System designs

Visual System Design Artifacts — checkpoint context

October 19, 2026 | Repository snapshot: 3c75abdc

Purpose and scope

Material Derailleur helps BWorks manage donated items and show donors their donation journey. This checkpoint documents the existing React, Express and PostgreSQL application at revision 3c75abdc, inspected October 19, 2026. It describes repository behavior and the checked-in local deployment configuration; no running production environment was verified.

Artifacts and rationale

Four visual artifacts accompany this one-page context: (1) a component and deployment diagram locates the browser, API modules, persistence and external services; (2) a six-entity data model shows ownership, optional program assignment and status history; (3) a swimlane flow exposes the current submission, review and donor experience; and (4) a proposed workflow makes the most useful refactoring decisions concrete. These views support onboarding and maintenance of an existing product.

Decisions and trade-offs

One Express backend keeps operations simple, but route handlers also contain database and integration logic. User accounts and Donor profiles are separate, matched by email rather than a foreign key. Structured ItemAttribute rows support varied donation types, while AI tags remain JSON metadata. Images live outside PostgreSQL. AI analysis and email currently run in request handlers, coupling response time to external services.

Current versus intended state

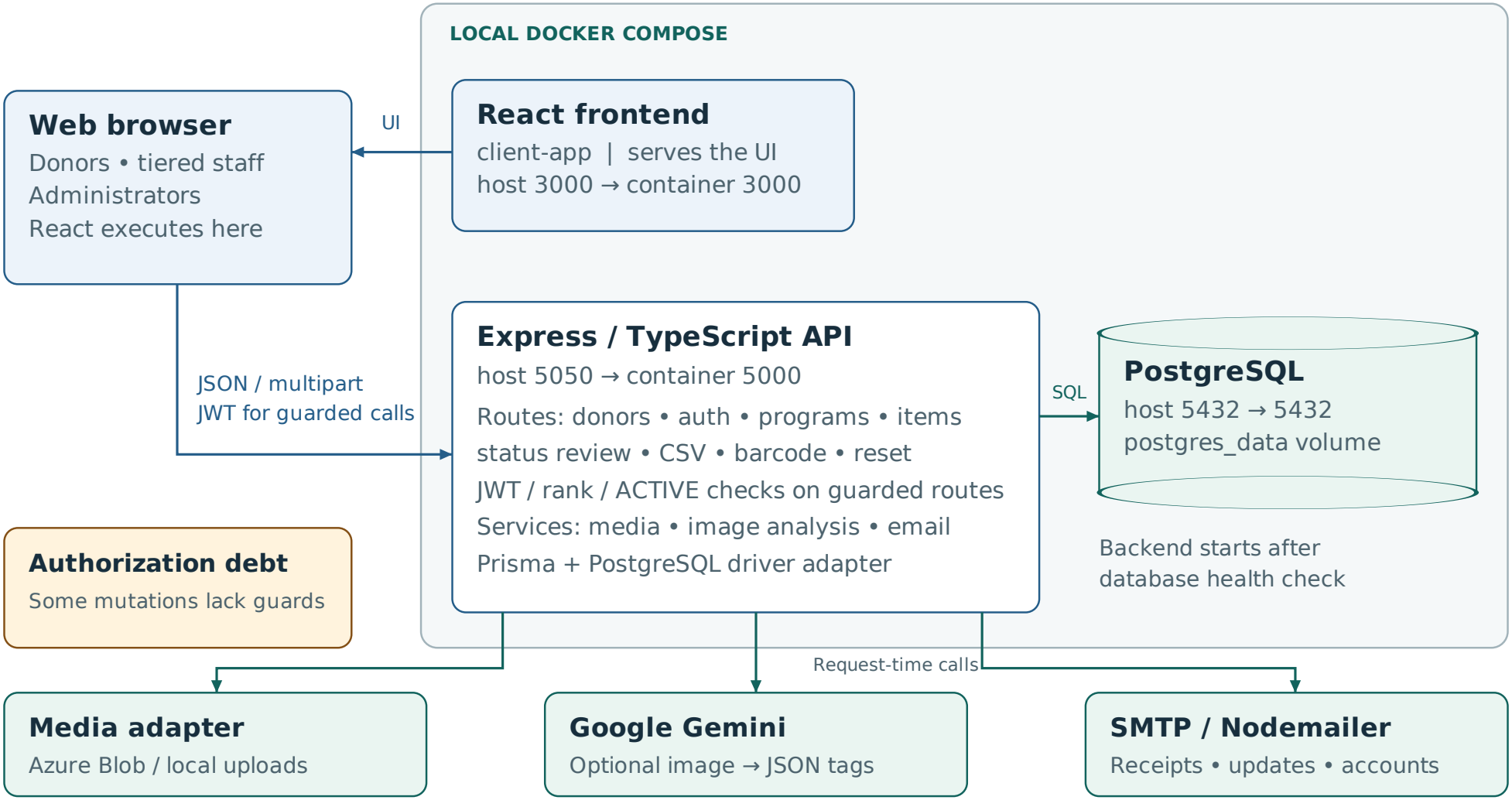
Today, submitting a status updates the item immediately and inserts pending history in a separate write. Approval controls donor timeline visibility in the UI and optional email; rejection deletes the history without restoring item state. The donor API still returns pending history. The proposed design stages updates, commits approved changes and an outbox event in one transaction, retains rejected records, filters donor responses on the server and records retryable notification delivery. Consistent authorization is a second proposed boundary improvement. These are recommendations, not implemented features or an agreed sprint commitment.

How this guides development

Use the architecture and schema views when changing integrations or migrations. Use the current workflow to reproduce inconsistent item/history behavior. Review the proposed approval semantics with BWorks before implementation, then migrate the review model and add transaction, rejection, donor-visibility and duplicate-notification checks. Revisit local media serving and development-only deployment settings before production use. Keep the Markdown diagrams, optional exports and source map aligned with code changes.

01 System architecture & local deployment

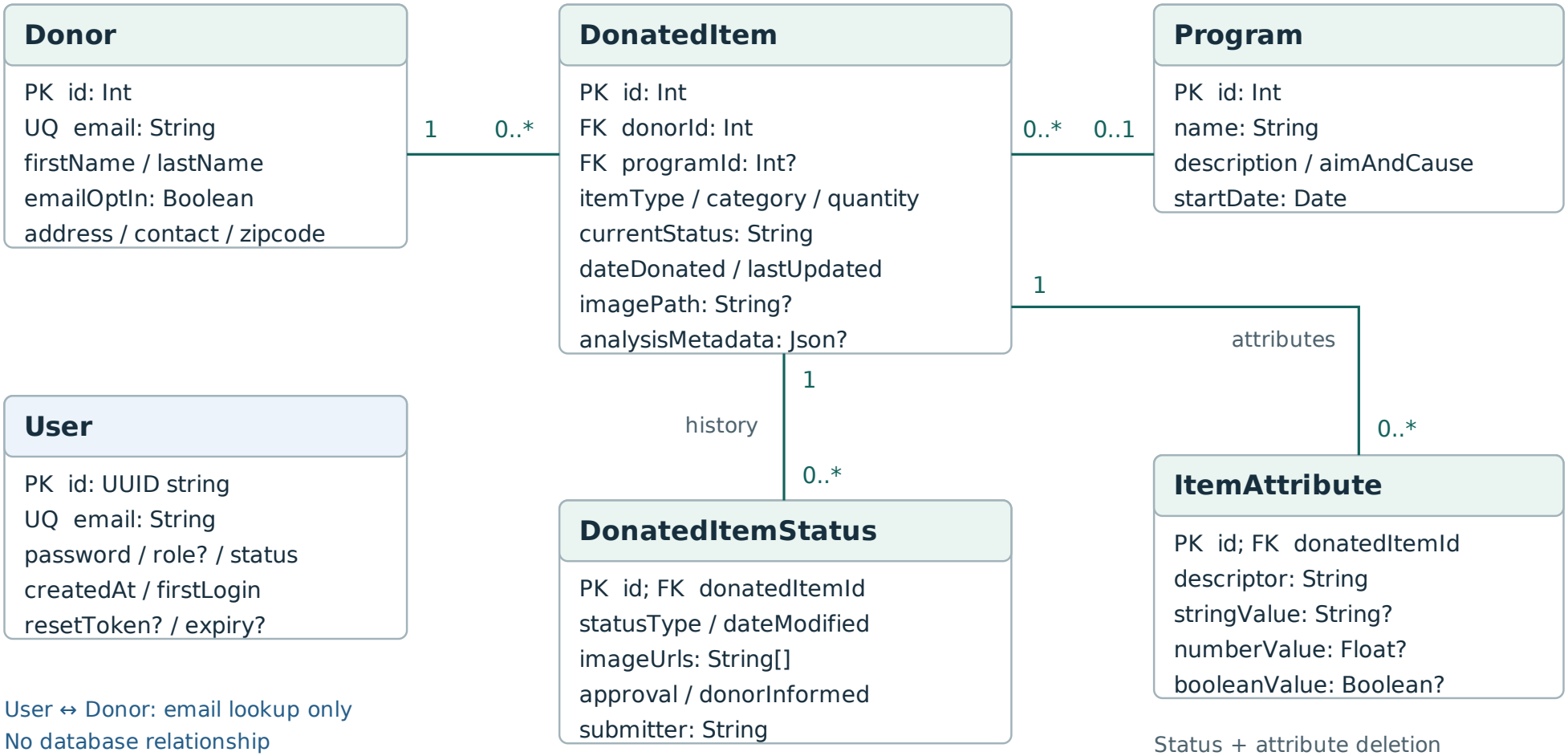
One application backend; three Compose services; external media, AI and email integrations.



Source: index.ts; prismaClient.ts; service modules; docker-compose.yml. Snapshot 3c75abdc. Not a verified production topology.

02 Data model & ownership

Entity relationships from the Prisma schema; representative fields keep the view readable.



User ↔ Donor: email lookup only
No database relationship

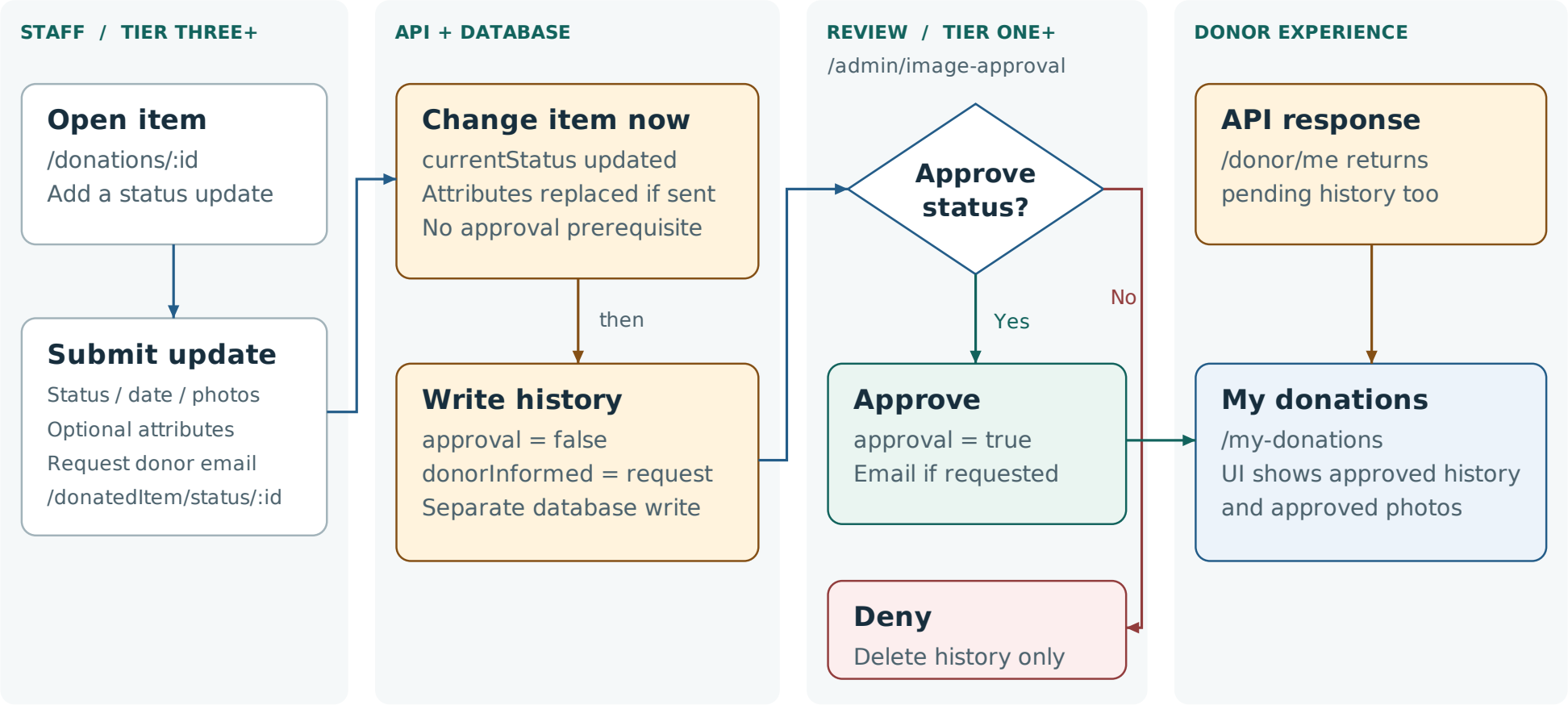
PK primary key • FK foreign key • UQ unique • ? nullable

Status + attribute deletion cascades from the item

Source: server/prisma/schema.prisma; donorRoutes.ts /me. Numeric multiplicities: 1 required, 0..1 optional, 0..* many. 3c75abdc.

03 Status submission → review → donor

Approval moderates history and update email; it does not gate the current inventory status.



Current-state consequences

Denying leaves item state changed. Email intent is not delivery proof. Business status values are not a sequential state machine.

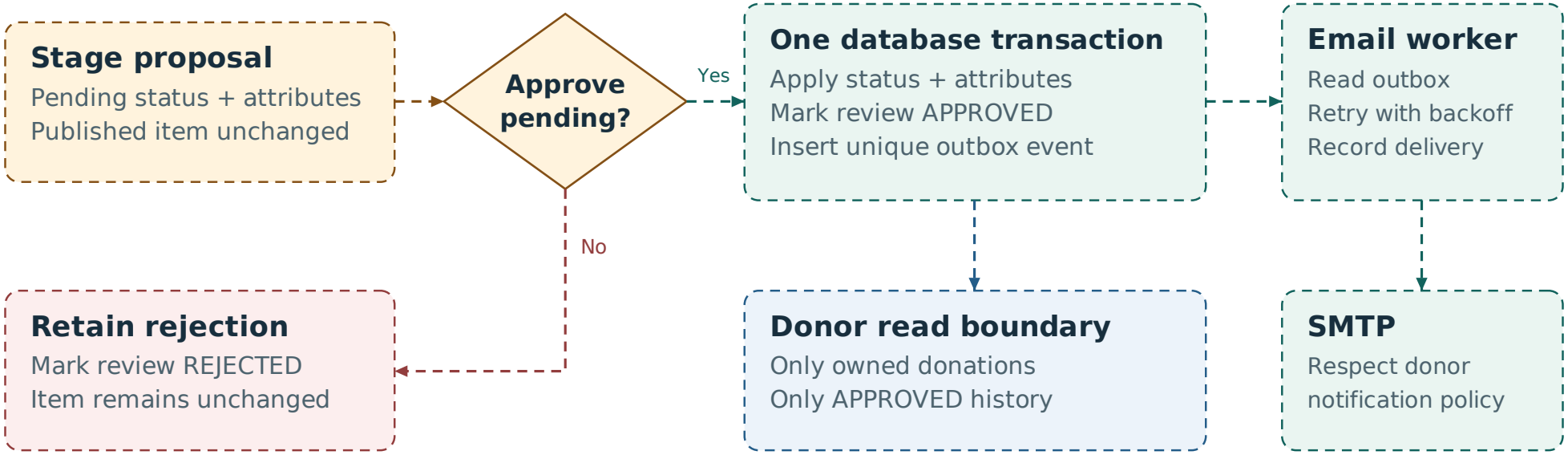
Source: donatedItemStatusRoutes.ts; donorRoutes.ts /me; DonorDonations.tsx; App.js. Tier One+ includes Admin. Snapshot 3c75abdc.

04 Proposed approval & delivery boundaries

Recommendation for BWorks review; new review records, transaction and outbox are not implemented.

Consistent API authorization

Verify access-token purpose + known claims • load existing ACTIVE user + current role • enforce donor ownership



Decisions before implementation

Confirm approval/publication semantics and email opt-in with BWorks; migrate existing pending history carefully.
Use a conditional PENDING → APPROVED transition and unique event key to prevent repeat approvals from resending.

Proposed only. Trade-off: adds review/outbox data and worker operations in exchange for consistent item history and retryable email. Dashed = proposed.